ADR-001 — Captura de pedidos com CDC (Debezium)

Contexto

A ShopVibe só enxerga seus pedidos no dia seguinte: um script copia as tabelas às 2h da manhã e o estoque é atualizado em lote a cada 4 horas. Na última Black Friday isso custou 300 pedidos cancelados por estoque desatualizado, R$ 180 mil em chargebacks de uma fraude descoberta tarde e carrinhos abandonados sem reação.

A empresa tem cerca de 15 mil pedidos por dia, com picos de 20 vezes esse volume (perto de 300 mil pedidos/dia). A diretoria quer painel ao vivo, alerta de fraude em segundos e estoque confiável. Os pedidos já nascem em um PostgreSQL usado pelo site e pelo app, e esse banco não pode ficar mais lento nem exigir mudanças nas aplicações em plena campanha.

Precisamos capturar cada pedido novo (RF01) em poucos segundos, sem perder nenhum e sem depender do relatório noturno.

Decisão

Capturamos os pedidos lendo o log de transações (WAL) do PostgreSQL com o Debezium, executado no Kafka Connect.

O PostgreSQL usa replicação lógica (`wal_level=logical`, plugin `pgoutput`) com uma publication restrita às tabelas `orders` e `order_items`. O Debezium faz um snapshot inicial e depois publica cada INSERT, UPDATE e DELETE como evento em um tópico do Kafka por tabela (`shopvibe.shopvibe.orders` e `shopvibe.shopvibe.order_items`). As tabelas usam `REPLICA IDENTITY FULL`, para que os eventos tragam a linha completa antes e depois da mudança.

O site e o app não mudam uma linha de código. A latência esperada entre o commit do pedido e o evento no Kafka é de poucos segundos, e a demonstração em vídeo mostra isso.

Alternativas consideradas

Nenhuma das alternativas abaixo entrega pedidos em segundos sem tocar na aplicação ou sobrecarregar o banco de vendas.

Alternativa	Por que não foi escolhida
Polling por `updated_at`	Gera consultas repetidas no banco de vendas, a latência é o intervalo do polling e DELETEs passam despercebidos
Triggers com tabela de auditoria	Acrescentam uma escrita dentro da transação de cada venda, no caminho crítico, e precisam de manutenção tabela a tabela
Dual-write (aplicação grava no banco e no	Se uma das duas gravações falhar, os dados

Alternativa	Por que não foi escolhida
Kafka)	divergem, e exige alterar o site e o app
Manter o ETL em lote	É o problema atual: a visibilidade chega com horas de atraso
Serviço gerenciado de CDC	Cobrado por uso e fora da restrição de usar apenas ferramentas open source

Consequências

Ganhamos latência de segundos sem mexer nas aplicações, ao custo de uma peça nova para operar e de um risco real sobre o disco do PostgreSQL.

Positivas

- Latência de segundos, sem polling.
- Carga baixa no banco de vendas: o Debezium lê o log, não consulta as tabelas.
- Captura todo INSERT, UPDATE e DELETE na ordem do commit, inclusive o que a aplicação esquecesse de publicar.
- O snapshot inicial permite carregar o histórico existente.

Negativas

- O slot de replicação retém WAL. Se o Debezium ou o Kafka Connect ficarem parados, o disco do PostgreSQL pode encher. É preciso monitorar o atraso do slot e limitar a retenção (`max_slot_wal_keep_size`).
- Exige `wal_level=logical` e `REPLICA IDENTITY FULL`, que aumenta o volume de WAL nas tabelas capturadas.
- Mais uma peça para operar (Kafka Connect), com entrega at-least-once: pode haver evento duplicado, tratado nas camadas seguintes.
- Os eventos acompanham o esquema das tabelas; mudanças de coluna precisam ser coordenadas. Um Schema Registry com Avro fica como evolução.
- No laboratório há uma única instância de cada componente; em produção seriam necessários standby do PostgreSQL e mais de um worker do Connect.